

Generative AI Categories: Text, Image, Video, and Audio



Xavier Morera

Helping developers create epic generative applications

www.xaviermorera.com / www.lupo.ai



The Categories

Text

Audio

Image

Video

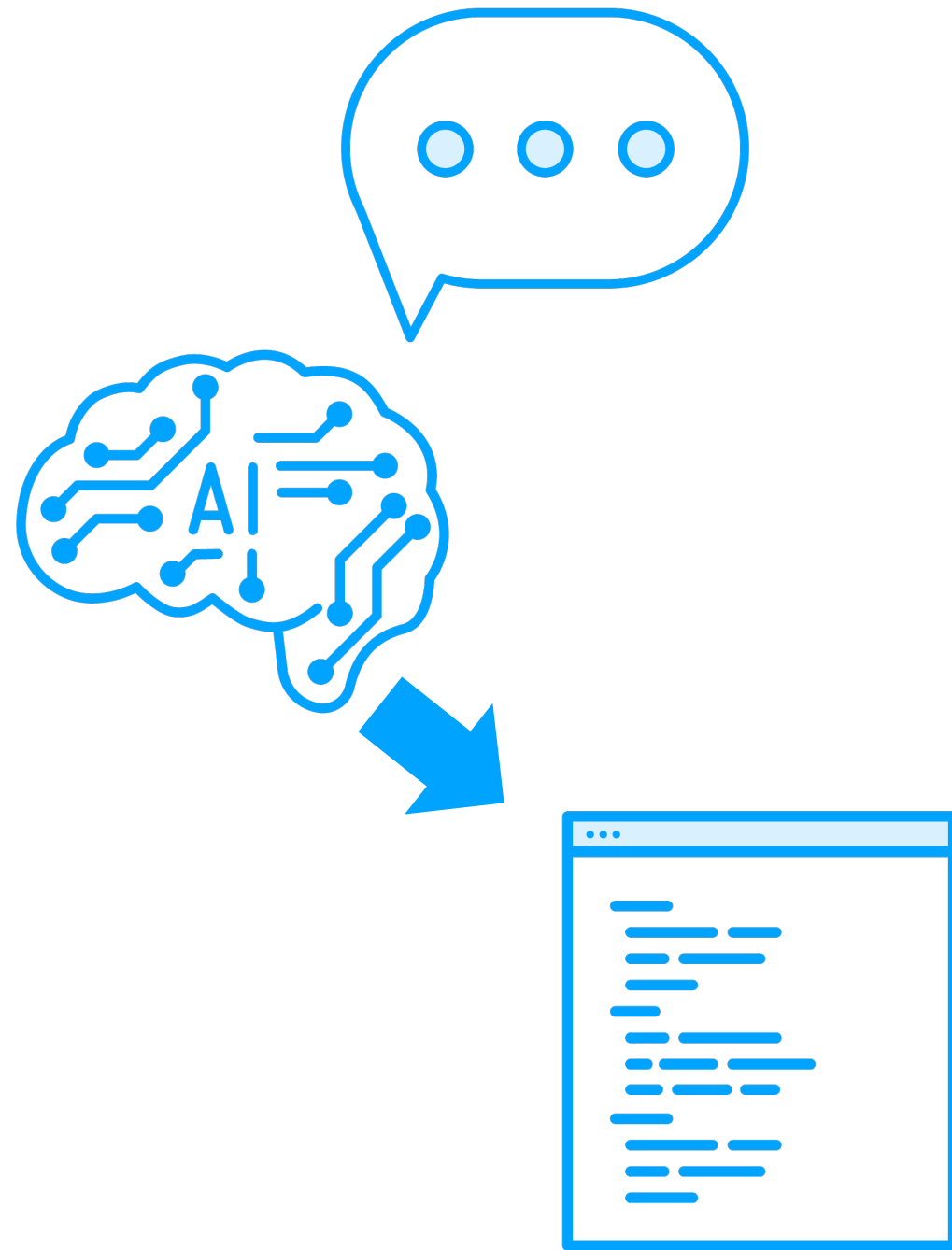




Text-based Generative AI



Text-based Generative AI



Process where an AI-driven system produces new textual content

- Looks and feels as if produced by a human

Generated based on a prompt

- Written in natural language



Natural language is the key



Natural Language is the Key



The use of natural language is the most important aspect of Gen AI

Ask questions (prompt) in the same way as you communicate with another human being

- And get an expert-level response

Called NLP or natural language processing

- No need to use a typical UI



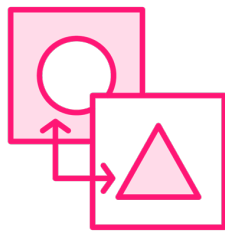
Applications of Text-based Gen AI



Content generation like blogs, stories, recipes, poems, and other forms of creative writing



Conversational agents or chatbots which are far superior to those chatbots where you get predefined menu options



Language translation, text summarization, text completion, transcription, and entity extraction



And more!



Types of Responses



Responses are based on the data on which models were trained

- Books, articles websites, social media posts...
- Publicly available data

When you use a text-based Gen AI platform

- Usually hard to know the training data used

Possible to train models on specific subsets of your data



The very high-level explanation of how LLMs work





A Large Language Model

It is like a "very powerful and complex autocomplete?"



How LLMs Work



LLMs generate responses based on the information they were trained

Predict one word at a time

- But on a much large scale
- With deeper understanding of the context





 chatgpt



 **chatgpt trained on copyright data**

 **chatgpt llm**

 chatgpt

 chatgpt 4

 chatgpt **app**

 chatgpt **login**

 chatgpt **detector**

 **chat gratis**

 chatgpt **español**

 chatgpt **que es**

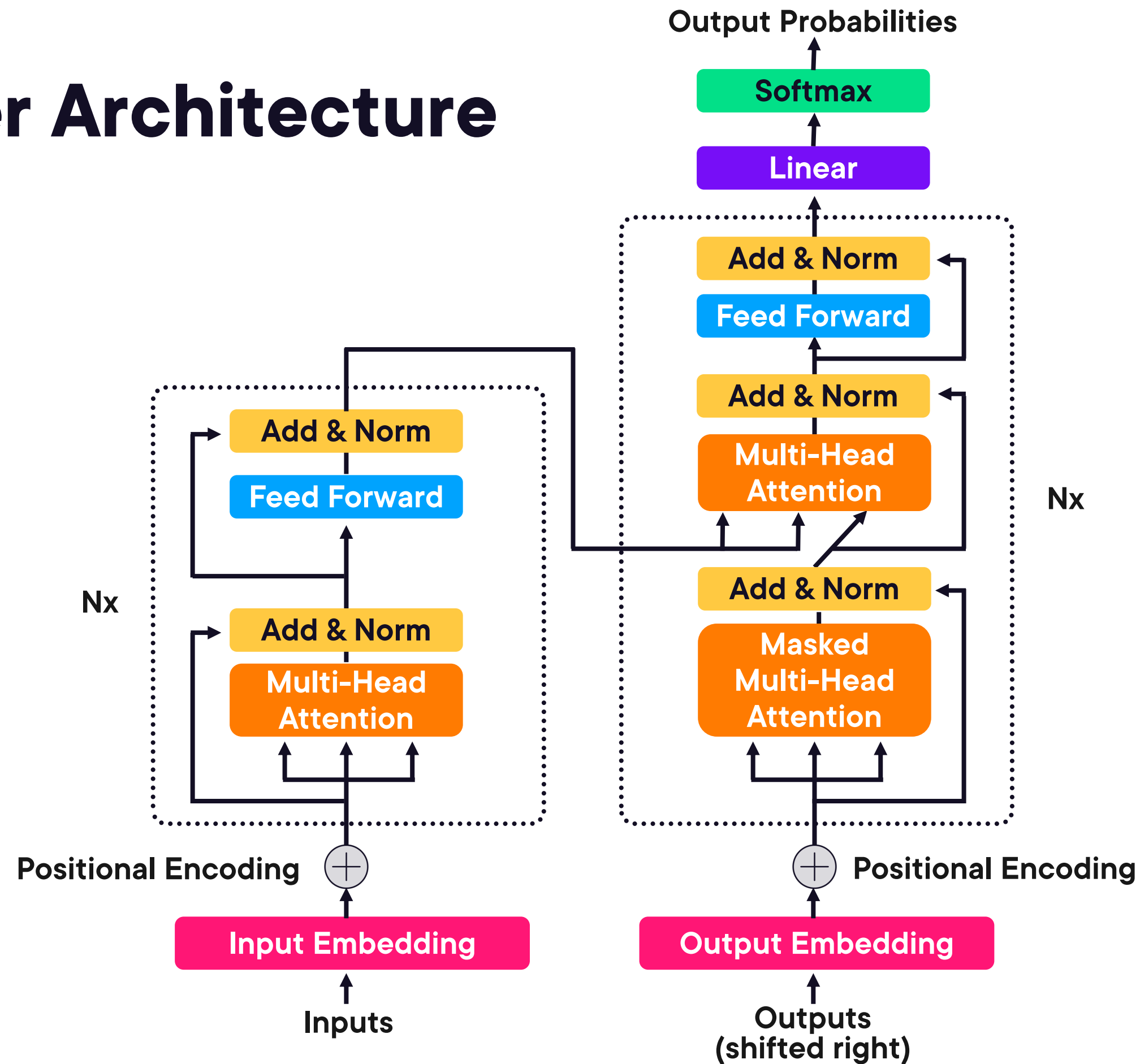
Google Search

I'm Feeling Lucky

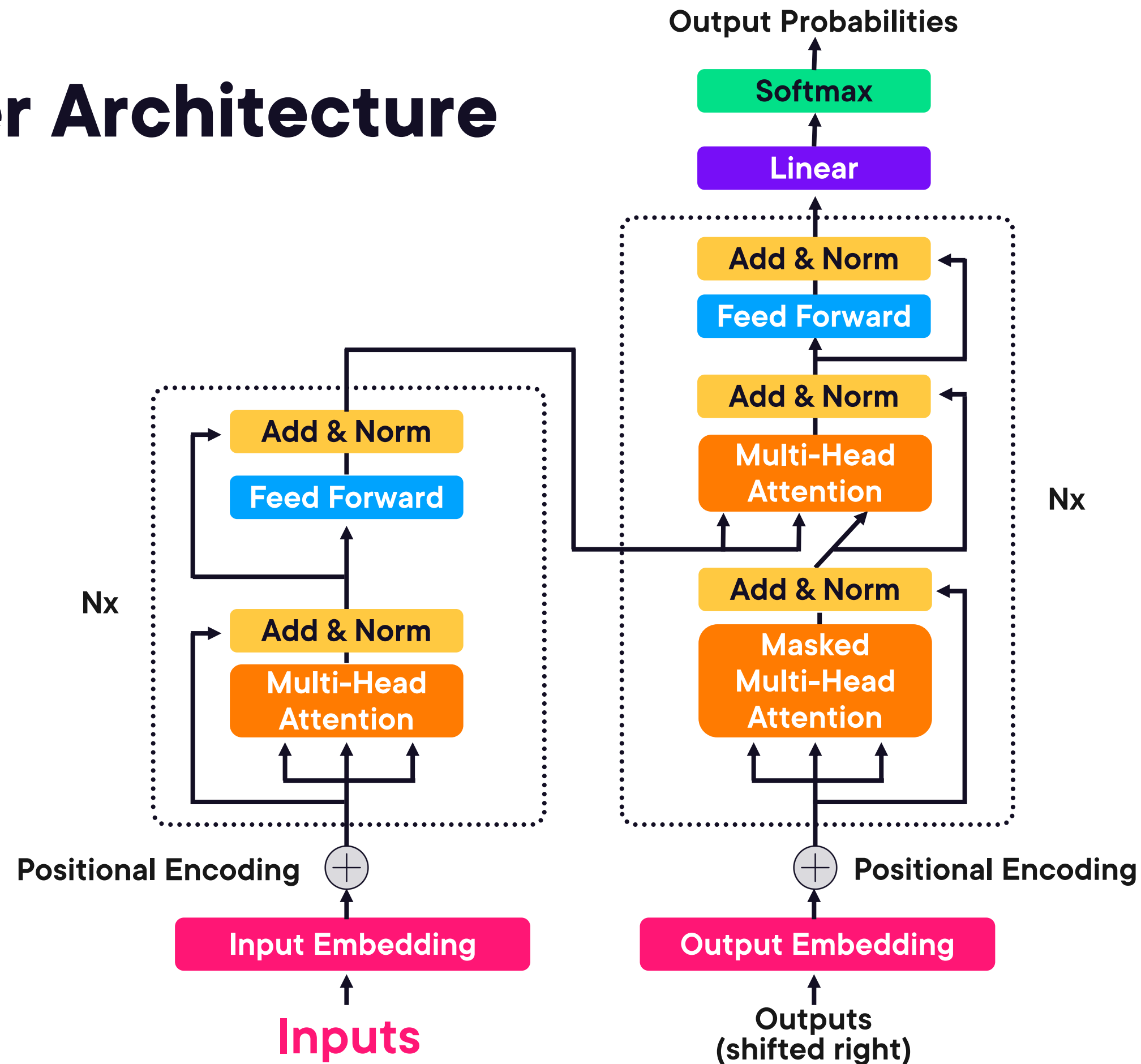
Costa Rica

[Report inappropriate predictions](#)

Transformer Architecture



Transformer Architecture



Input a.k.a. the Prompt

"Write a short story on a traveling computer programmer"



**You**

Write a short story on a traveling computer programmer

**ChatGPT**

Once upon a time, in a world where technology intertwined with every aspect of life, there was a young computer programmer named Alex. Unlike most programmers who were content with their sedentary lifestyles, Alex had an insatiable wanderlust burning within.

From a young age, Alex was fascinated by computers. While other kids played outside, Alex was indoors, tinkering with code and building programs. As Alex grew older, the desire to explore the world outside the confines of their computer screen grew stronger.

One day, Alex made a bold decision. They packed up their laptop, a few changes of clothes, and set off on a journey across the globe. With nothing but their programming skills and a thirst for adventure, Alex embarked on an extraordinary journey.

Their first stop was in a bustling city where they found a cozy café with free Wi-Fi. With their laptop open, Alex dove into the local programming community, exchanging ideas and collaborating on projects. They quickly became known as the traveling programmer, a nomad of the coding world.



From there, Alex journeyed to remote villages where internet access was scarce. Undeterred, they

Message ChatGPT...




```
{
  "role": "user",
  "content": "Write a short story on a traveling computer programmer"
},
{
  "role": "assistant",
  "content": "Once upon a time, there was a talented computer programmer named Alex who had a passion for coding and exploring new places. Alex had always dreamt of combining their love for technology with their love for travel. So, one day, they decided to embark on a journey around the world as a traveling computer programmer.\n\nArmed with their trusty laptop and a thirst for adventure, Alex set off on their first destination – a bustling city known for its innovative tech scene. They quickly found freelance work as a programmer, helping local businesses with their software needs while immersing themselves in the culture and sights of the city.\n\nAs Alex traveled from one place to another, they encountered various challenges and opportunities that tested their skills and adaptability. From coding in a cozy cafe in Paris to troubleshooting a bug on a beach in Thailand, Alex's journey was filled with excitement and learning experiences.\n\nAlong the way, Alex made friends with fellow programmers and tech enthusiasts from different parts of the world, exchanging ideas and collaborating on projects that pushed the boundaries of their creativity. They also discovered the beauty of remote work, allowing them to code from breathtaking landscapes and inspiring locations.\n\nThrough their travels, Alex not only honed their programming skills but also gained a deeper understanding of the diverse ways in which technology could impact people's lives around the globe. They realized that being a traveling computer programmer was not just about writing code but also about connecting with others and making a positive difference in the world through their work.\n\nAnd so, Alex continued their journey as a traveling computer programmer, eager to explore new horizons, meet new challenges, and spread the joy of coding wherever they went. Armed with their laptop and a spirit of adventure, Alex was ready to code their way through the world, one destination at a time."
}
```

Response:

Once upon a time, there was a talented computer programmer named Alex who had a passion for coding and exploring new places. Alex had always dreamt of combining their love for technology with their love for travel. So, one day, they decided to embark on a journey around the world as a traveling computer programmer.

Armed with their trusty laptop and a thirst for adventure, Alex set off on their first destination – a bustling city known for



Tokenizer

Learn about language model tokenization

OpenAI's large language models (sometimes referred to as GPT's) process text using **tokens**, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

It's important to note that the exact tokenization process varies between models. Newer models like GPT-3.5 and GPT-4 use a different tokenizer than previous models, and will produce different tokens for the same input text.

GPT-3.5 & GPT-4

GPT-3 (Legacy)

Write a short story on a traveling computer programmer

Clear

Clear results

[GPT-3.5 & GPT-4](#)[GPT-3 \(Legacy\)](#)

Write a short story on a traveling computer programmer

[Clear](#)[Show example](#)**Tokens****9****Characters****54**

Write a short story on a traveling computer programmer

[Text](#)[Token IDs](#)

A helpful rule of thumb is that one token generally corresponds to ~4 characters of text for common English text. This translates to roughly $\frac{3}{4}$ of a word (so 100 tokens $\approx$ 75 words).



GPT-3.5 & GPT-4 **GPT-3 (Legacy)**

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🖐

Sequences of characters commonly found next to each other may be grouped together: 1234567890

Clear

Show example

Tokens

57

Characters

252

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🖐

Sequences of characters commonly found next to each other may be grouped together: 1234567890

Text

Token IDs

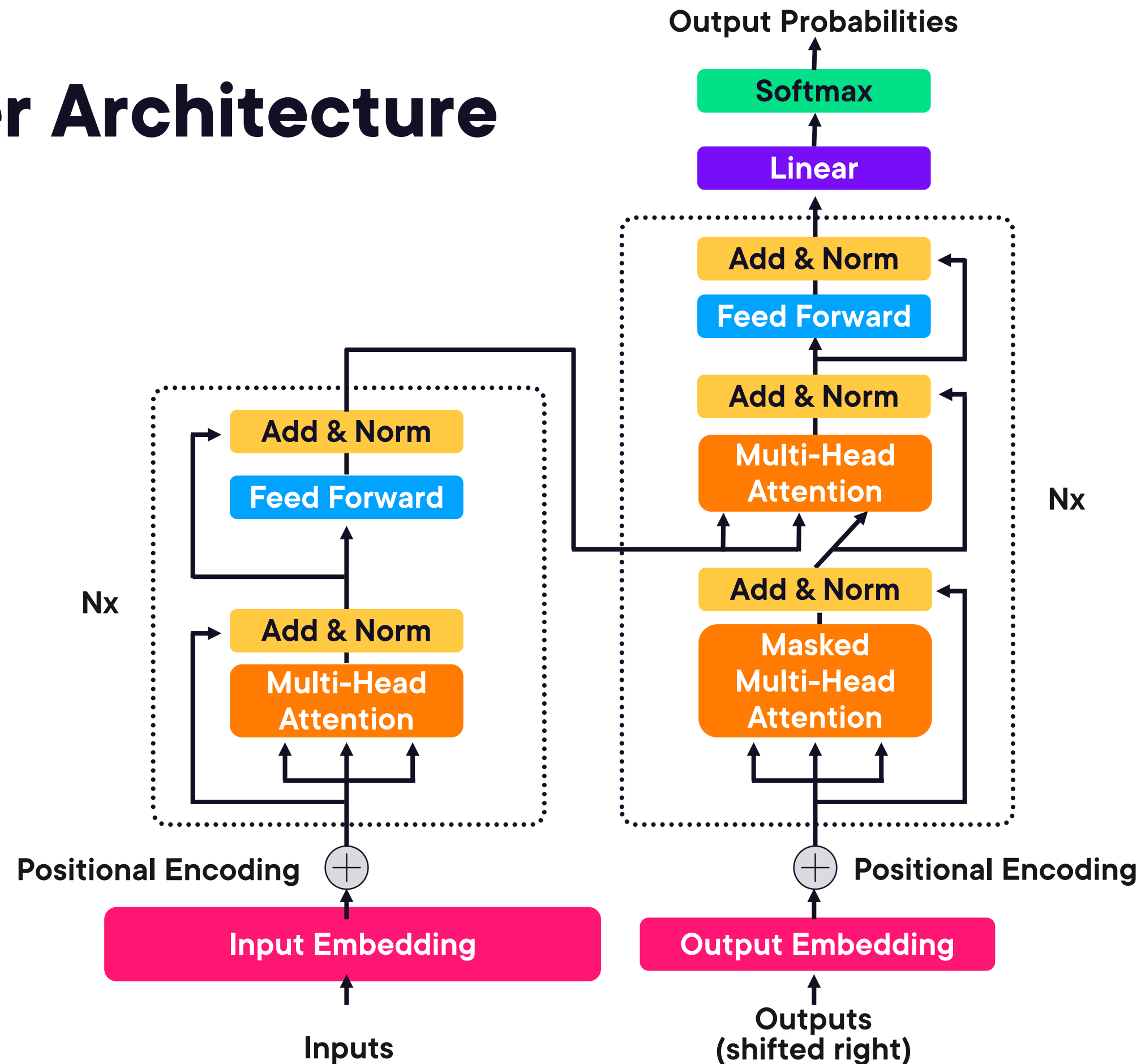
A helpful rule of thumb is that one token generally corresponds to ~4 characters of text for common English text. This translates to roughly $\frac{3}{4}$ of a word (so 100 tokens $\approx$ 75 words).

```
[9]: response = client.embeddings.create(
      input="Write a short story on a traveling computer programmer",
      model="text-embedding-3-small"
    )
```

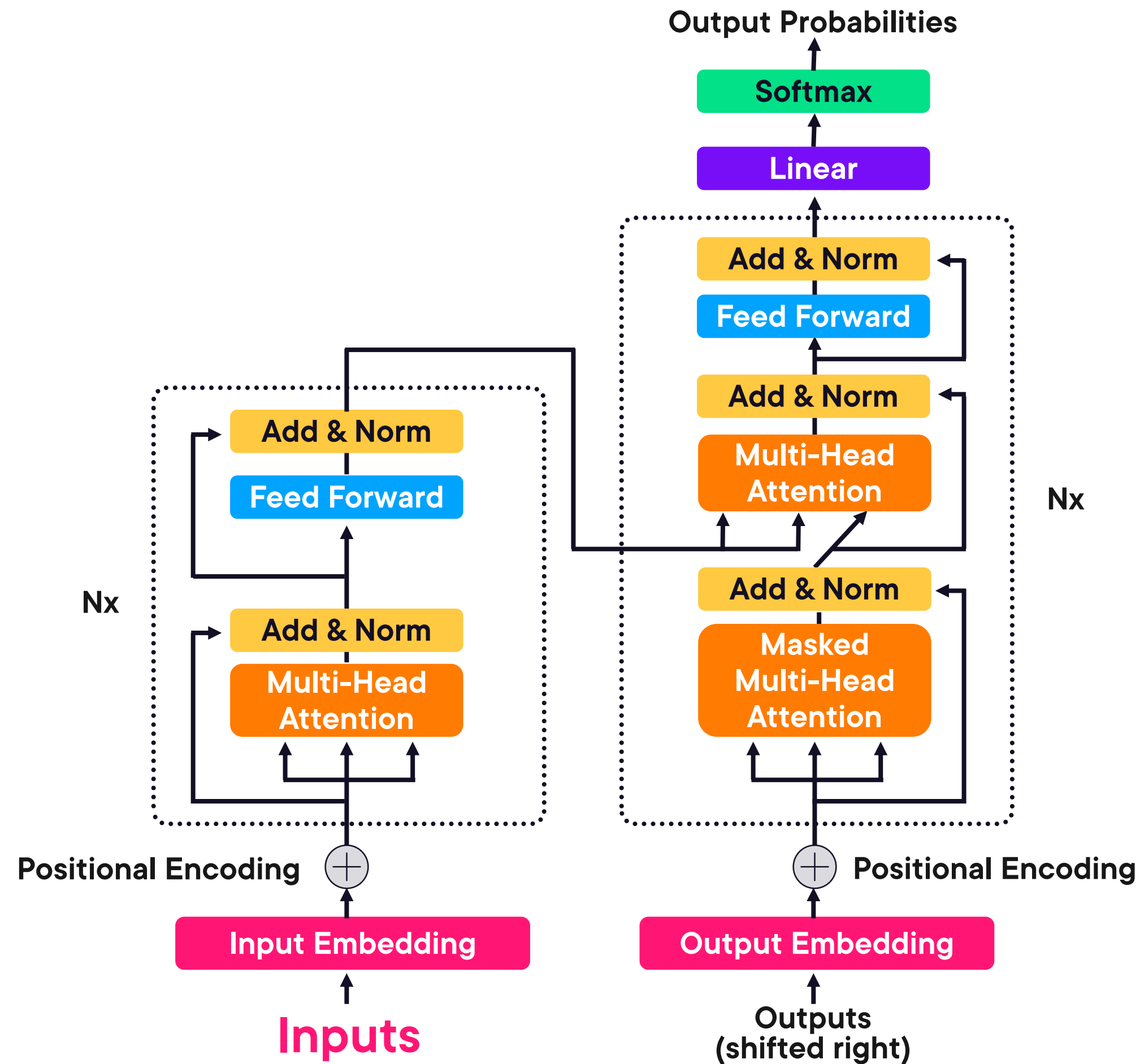
```
[10]: print(response.data[0].embedding)
```

```
[-0.011, -0.03158333, 0.001484375, -0.010618055, -0.017125, -0.06411111, 0.03888889, 0.03433333, 0.008520833, 0.009180555, 0.0
058055557, -0.009180555, -0.018555555, 0.00044487847, 0.015819445, -0.0019357639, -0.0033159722, -0.011083334, -0.021513889, 0.
021930555, -0.009090277, 0.03708333, 0.01913889, 0.020805556, -0.0047847223, 0.008486111, -0.0255, 0.02663889, 0.061611112, -0.
064, 0.0044131945, -0.011805556, 0.017958334, -0.013020833, 0.046916667, 0.0113194445, 0.018097222, -0.015652778, 0.04963889,
0.002248264, -0.015805556, -0.036194444, 0.029055556, 0.026944444, -0.029305555, 0.02398611, 0.00659375, 0.0060729166, 0.030833
334, 0.042944446, -0.036, 0.017833333, 0.00975, 0.0055902777, -0.03425, -0.018027777, -0.0285, -0.0027413194, 0.012680556, -0.0
20083332, 0.053916667, -0.03452778, 0.011854167, 0.016125, -0.020444445, 0.002684028, -0.032361113, -0.0037534721, -0.01947222
3, -0.016236112, 0.042805556, 0.053055555, -0.0061041666, 0.025180556, 0.00052300346, 0.00045833332, -0.03252778, 0.006003472,
0.0061284723, -0.0041770833, -0.004138889, -0.036944445, 0.027375, -0.019861111, -0.027319444, 0.0058854166, -0.0555, -0.054638
89, 0.0008606771, -0.04763889, 0.021791667, 0.04325, 0.044083335, 0.023486111, 0.056138888, 0.0002078993, -0.009875, 0.0273888
9, 0.008173611, 0.019208333, 0.0111875, -0.018555555, 0.002340278, 0.045555554, 0.048583332, 0.029444445, -0.023416666, 0.02738
889, -0.028666666, 0.016486112, -0.11744444, -0.012277778, 0.016402777, -0.034722224, -0.026263889, 0.044805557, 0.071777776, -
0.04227778, 0.02048611, -0.045027778, -0.030277777, 0.019736111, -0.0003643663, -0.020916667, -0.009888889, -0.037722223, -0.01
7430555, -0.031833332, -0.001736111, -0.010180555, 0.031527776, -0.041972224, 0.022569444, -0.035194445, -0.016319444, -0.0062
395833, -0.009618056, -0.022736112, -0.04025, 0.03286111, -0.029222222, 0.05827778, -0.037388887, 0.0053715277, -0.0026666666,
0.037805557, 0.016541667, 0.0705, 0.009854167, 0.018472223, -0.018111112, -0.0136875, -0.008875, 0.019902777, 0.09466667, -0.02
1194445, -0.018097222, -0.008256945, -0.03538889, 0.0077708336, -0.04227778, -0.014104167, -0.034944445, -0.017930556, 0.003743
0555, -0.032194443, 0.017069444, 0.03425, -0.06277778, -0.03358333, 0.006361111, -0.06838889, -0.045833334, 0.02175, 0.04541666
```

Transformer Architecture



Processing



Prediction



Based on the processed embeddings

- From the prompt tokens

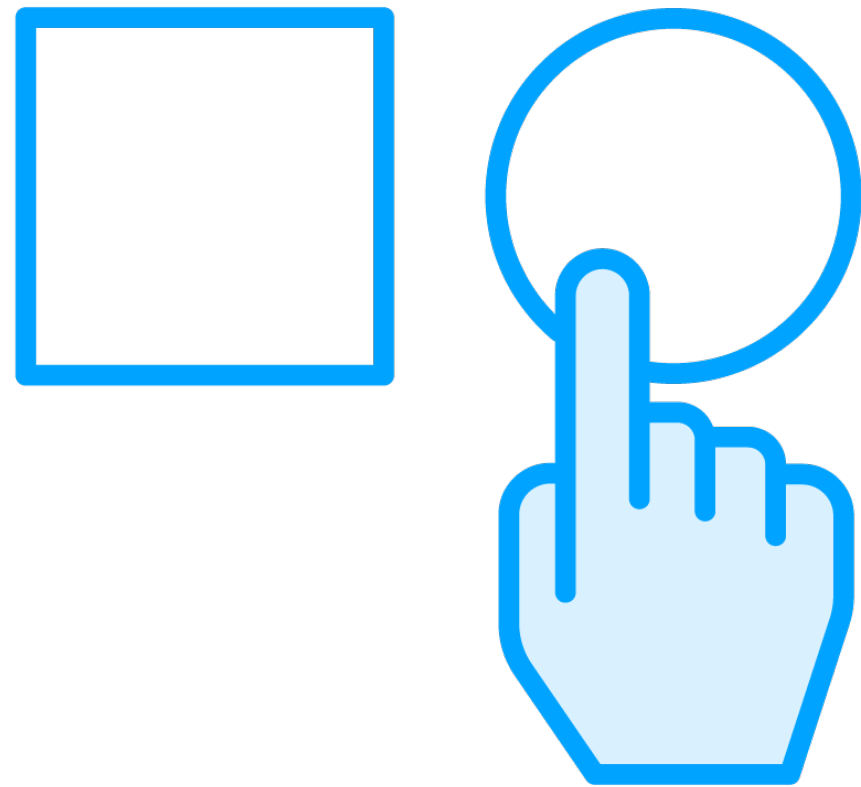
Model generates predictions for the next token or sequence of tokens

Transformer uses self-attention to understand the context

- Takes a look at all the words in the sentence
- This is what makes it superior



Sampling or Beam Search



Transformer uses several approaches to select the words

- From the predicted distribution of tokens

Can use beam search

- Looks at the probability of a larger set of tokens and picks the best one

Output

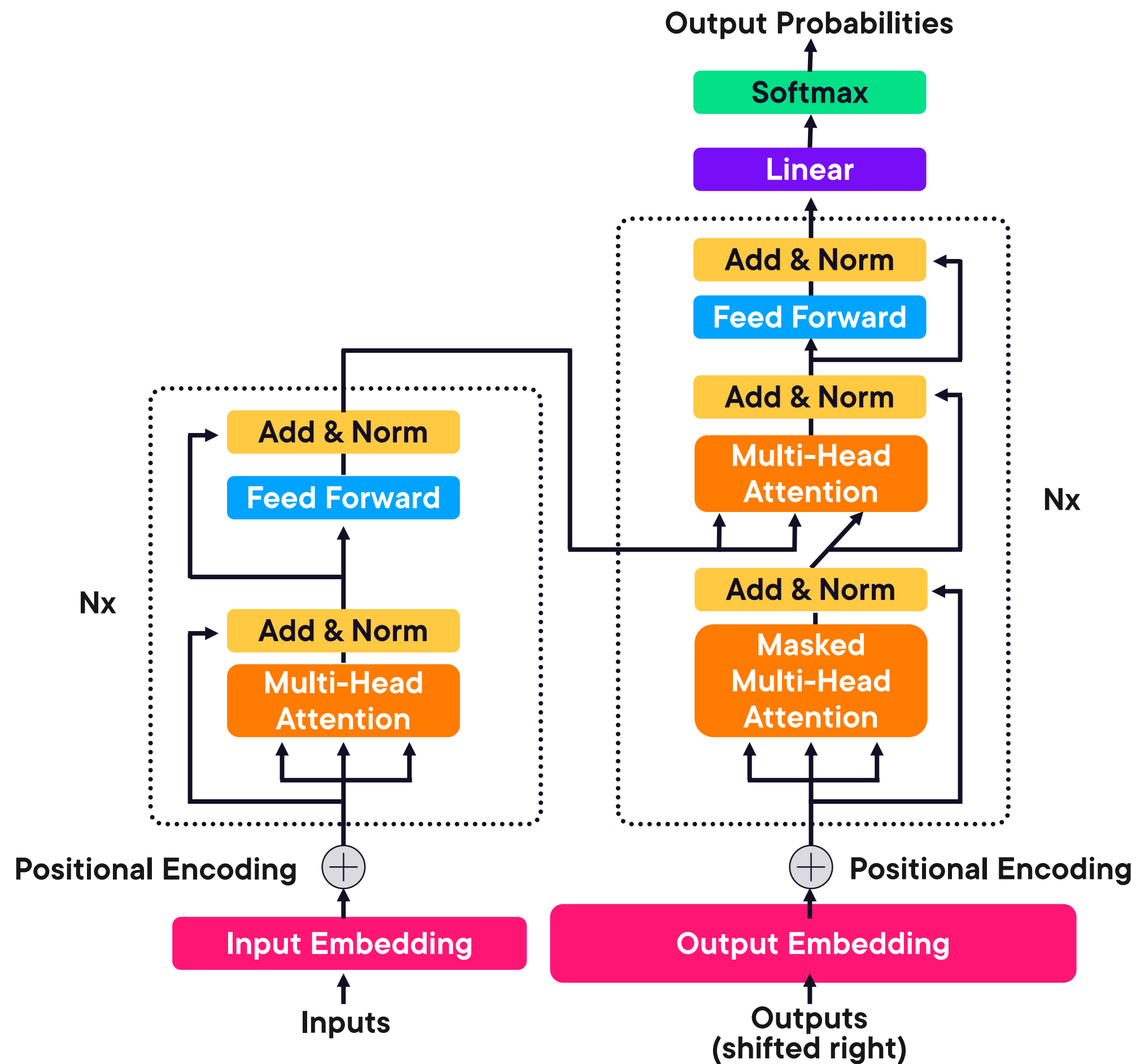


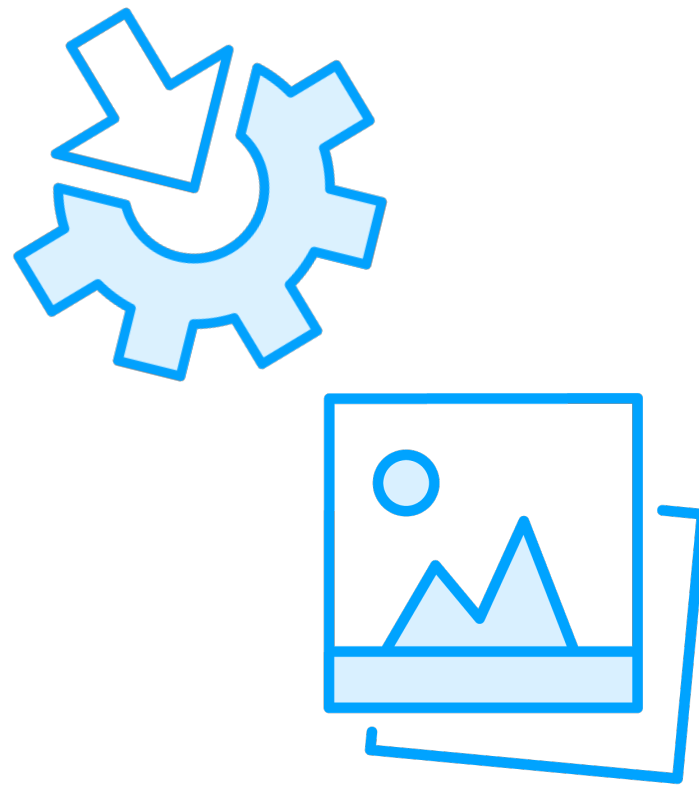


Image-based Generative AI



Image-based Generative AI

INPUT



AI models are used to generate new images

- Based on a text prompt

Results can vary

- From realistic photographs
- To artistic creations
- And everything in between

Multiple services available

- You can also train your own models





I've created a photorealistic image that captures the essence of a 45-year-old male riding a road bike up a steep hill in the Italian mountains. The image showcases the rugged terrain, the lush landscapes, and the determination of the rider as he faces the long and hard road ahead.



Message ChatGPT...

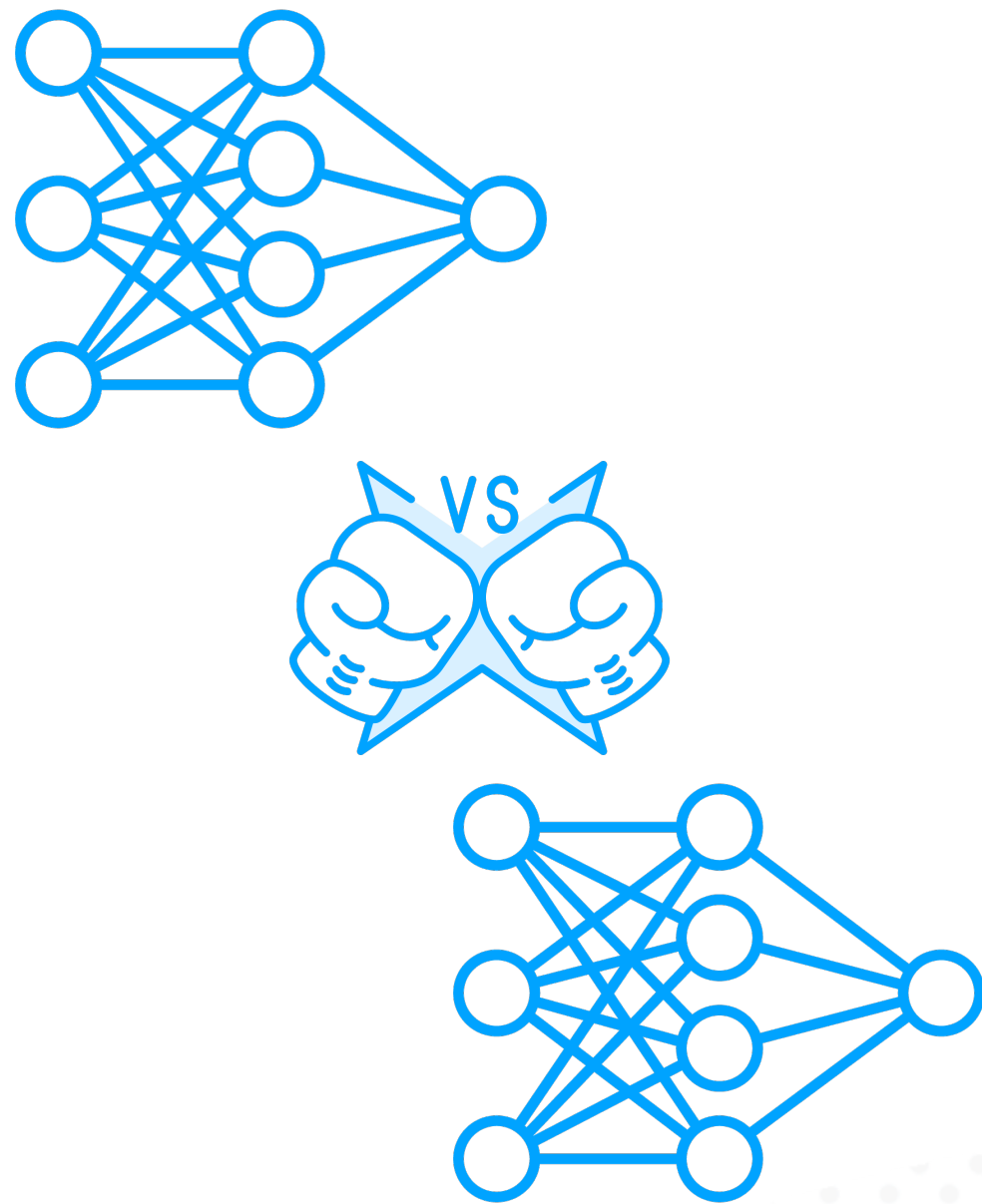


[illegible]

Types of Models Available to Generate Images



1. Generative Adversarial Networks (GANs)



Consists of two neural networks

- Generator and Discriminator

Trained to compete against each other

- In a competitive setup

Generator tries to create images that are indistinguishable from real images

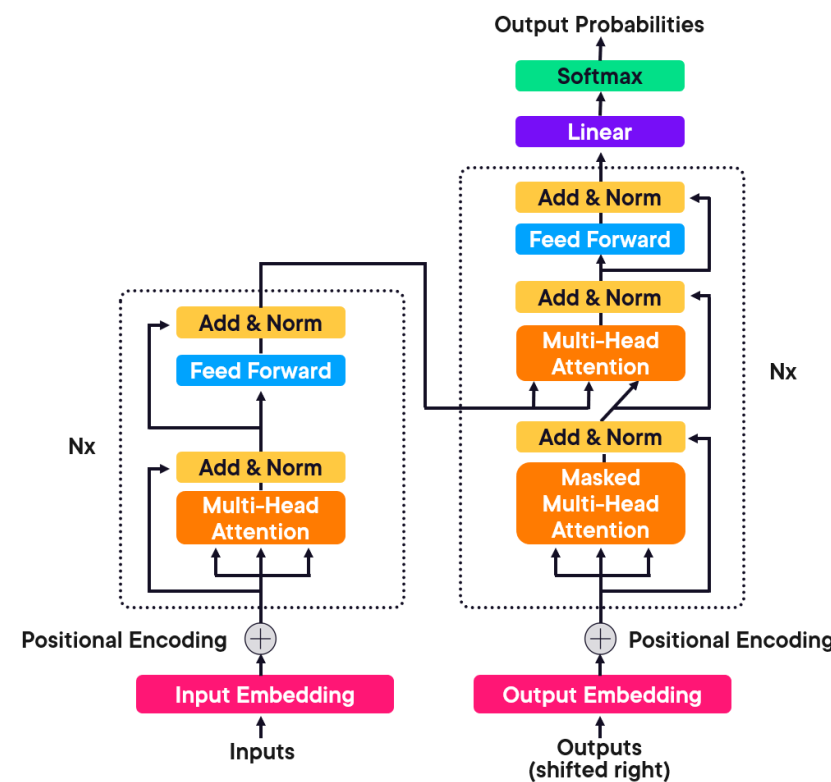
Discriminator evaluates to determine if they are real or not

Process continues until discriminator can't easily classify as fake





2. Transformers



Same type of model as used for text generation

- Adapted for image generation

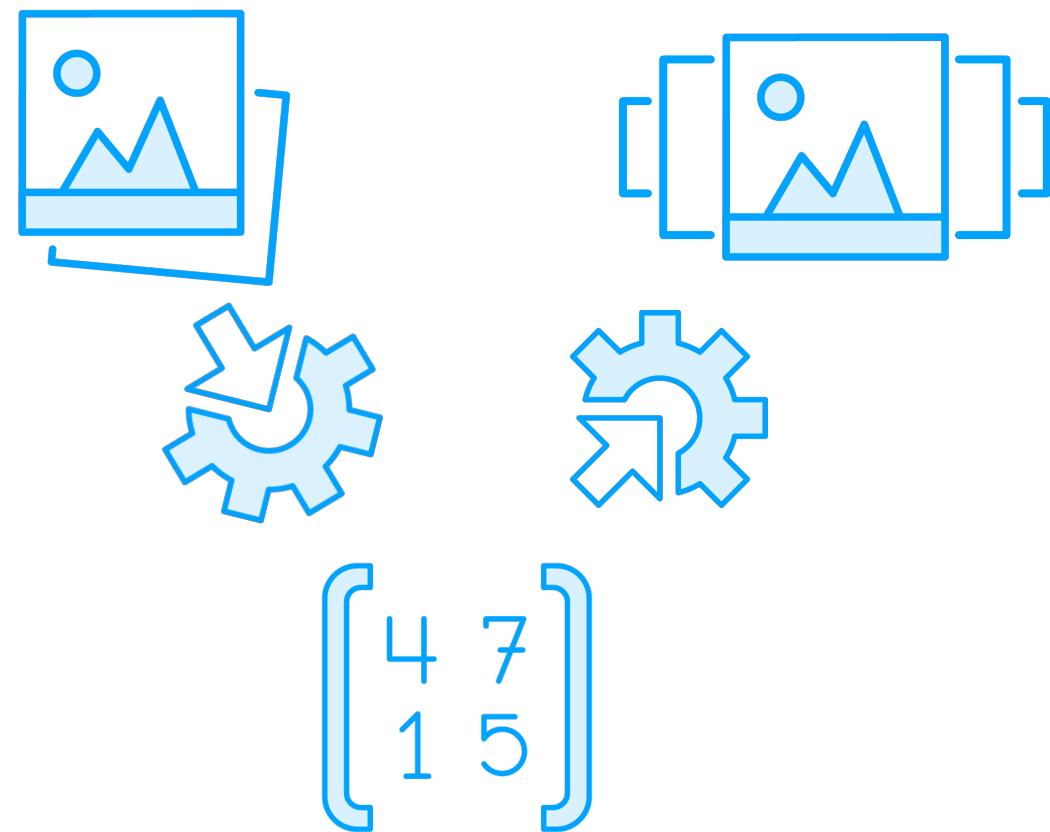
Very good at handling relationships between different parts of the image

Use attention mechanisms to focus on different part of the input data

- Allows creating highly detailed images



3. Variational Autoencoders (VAEs)



Designed to encode input data

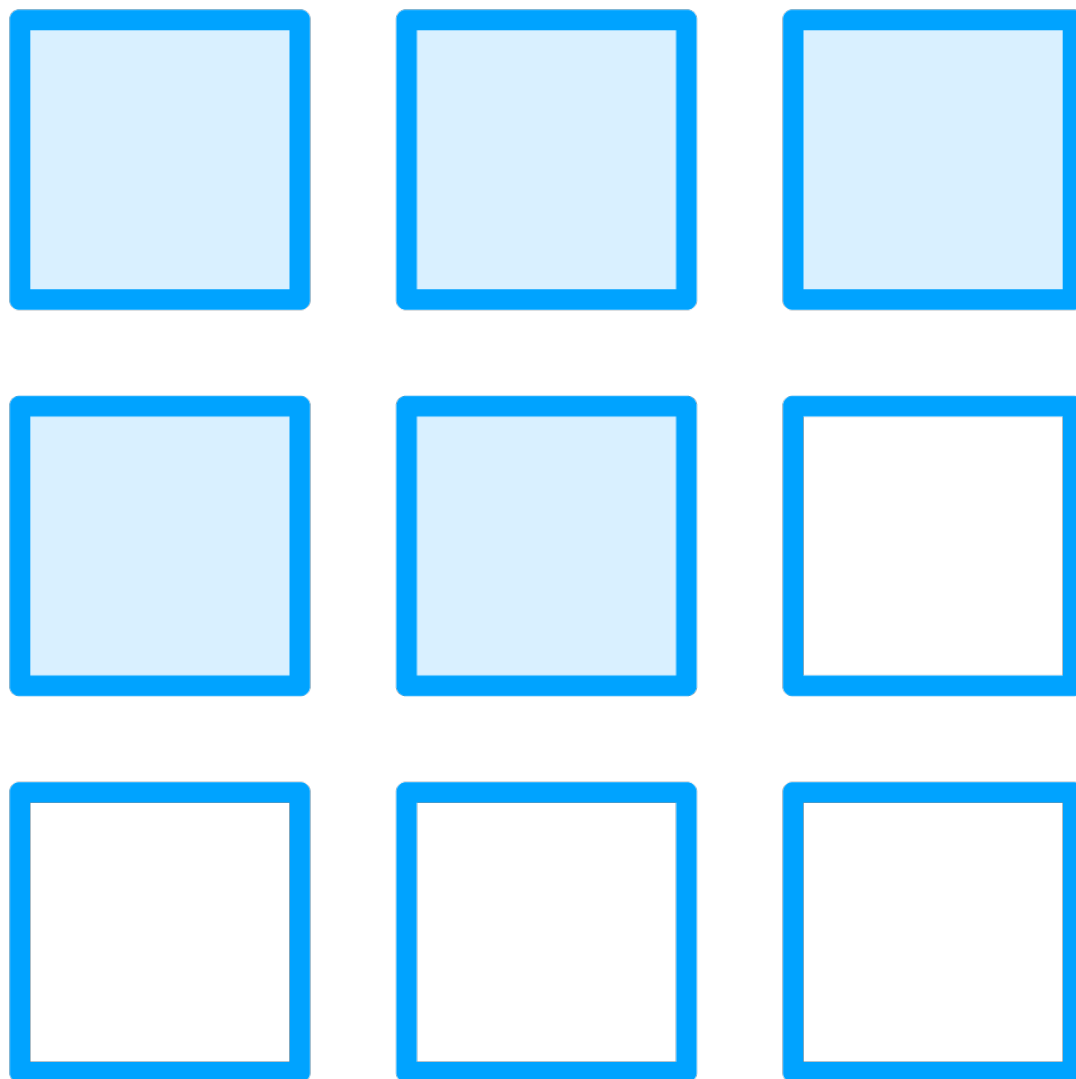
- Into a compressed representation
- In a latent space

Then decode back into the original format

New images with shared characteristics can be created

- But they are not identical

4. AutoRegressive Models



Generate images pixel by pixel

Treating the problem as a sequence generation task

- Each pixel is predicted based on the values of previous pixels

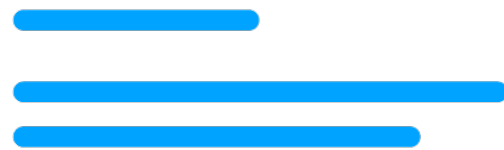
Allows for the creation of detailed and coherent images, over time

Typically use RNNs or CNNs to capture dependencies between pixels

Just like a printer



5. Diffusion Models



Start with random noise

Start refining the image in an iterative way

- Computationally expensive

End result is a coherent image

Created guided by a neural network

Can generate high-quality, high-resolution images in diverse styles and content



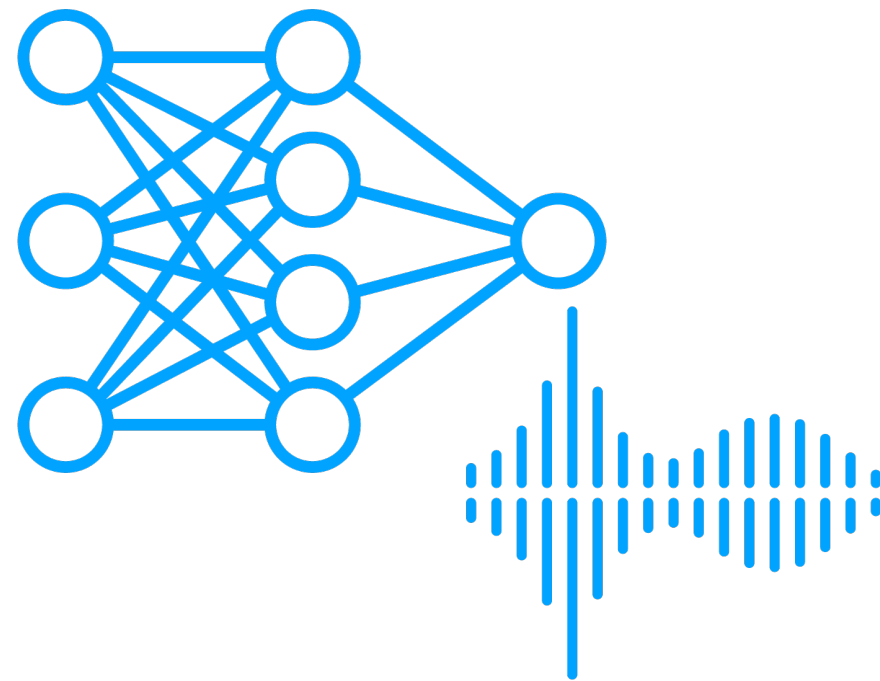
**Image-based generative AI
is a rapidly advancing field.**



Audio-based Generative AI



Audio-based Generative AI



Create new audio content

- Music production, voice synthesis, sound effects, and more
- Mimics real-world audio

Many challenges and considerations

- Copyright issues, bias and diversity, authenticity, and more



Types of Models Available

**Generative
Adversarial
Networks**

**Variational
Autoencoders**

Autoregressive

Transformers

**Recurrent Neural
Networks**

**Convolutional Neural
Networks**



The Process

**Collect large
dataset of
audio files**

**Data
collection**

**Raw audio
processed to
extract features**

**Feature
extraction**

**Model learns to
predict audio
samples or
sequences**

**Model
training**

**Generate new
samples that
have temporal
coherence**

**Synthesis and
post-processing**



×
Genre
Mood
Theme
Length **3:00**
Tempo
Instruments

+ Video Preview

Hip Hop

Trap

Drill

R&B

Jersey Club

Latin

Acoustic

Rock

Ambient

Beats

Drum n Bass

Electro & Dance

Funk

House

Techno & Trance

Tokyo night pop

Pop

Lofi Hip Hop

World

Electronica

Orchestra

Tropical House

Afrobeats

Phonk

Christmas

Hip Hop ×



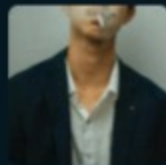
001

Hip Hop

Dark, Laid Back, Hopeful, Sentimental,...

100 BPM

3:04



002

Hip Hop

Dark, Heavy & Ponderous, Sad, Hopef...

145 BPM

3:00



003

Hip Hop

Dark, Laid Back, Heavy & Ponderous, ...

140 BPM

3:00



004

Hip Hop

Dark, Angry, Heavy, Mysterious, Fear, ...

140 BPM

3:00



005

Hip Hop

Heavy, Dreamy, Hopeful, Glamorous, P...

90 BPM

3:03



006

Hip Hop

Dark, Sad, Laid Back, Sentimental, Gla...

145 BPM

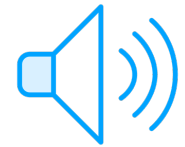
3:00



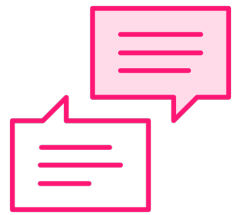
Applications



Music generation



Sound effects



Text-to-speech and speech-to-text



Voice cloning



And more





Video-based Generative AI



Creating an Image

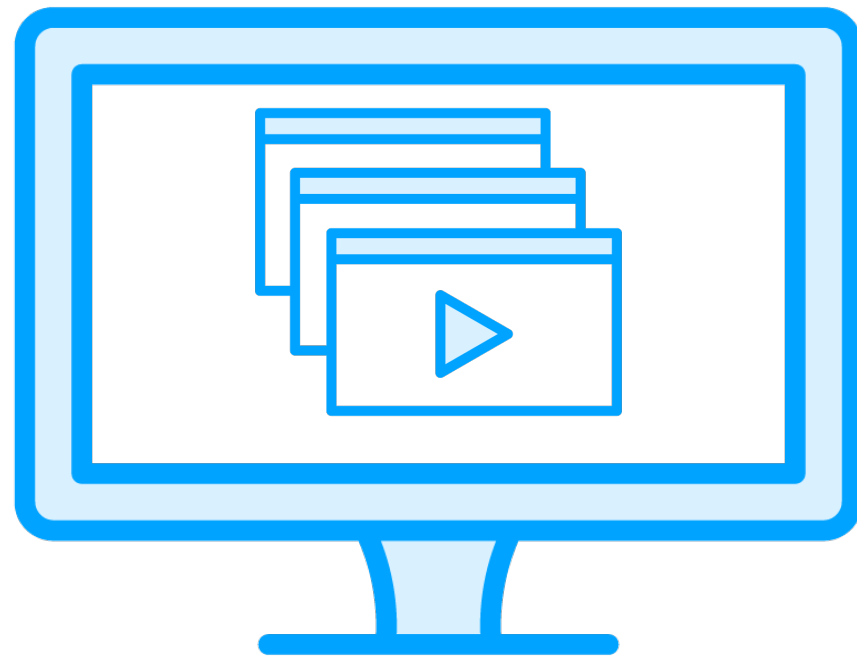


Creating an image from a prompt is by itself an astounding accomplishment (IMHO)

- However, it is a static and 2-dimensional array of pixels
- Does not take into consideration time or motion



Creating a Video



Creating a video requires creating a series of images that change over time

- Sequence

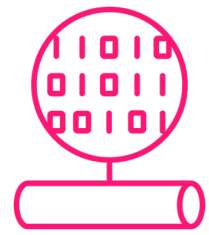
Requires understanding of the contents of the image

- Considering time
- Movements must be consistent with the real world

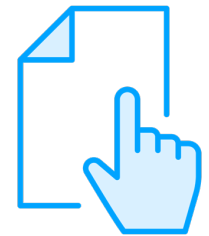
Very complex and computationally expensive



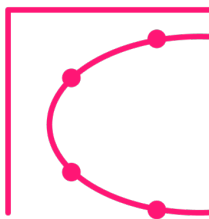
The Process Is Similar to Images but with a Difference



Data collection



Preprocessing and feature extraction



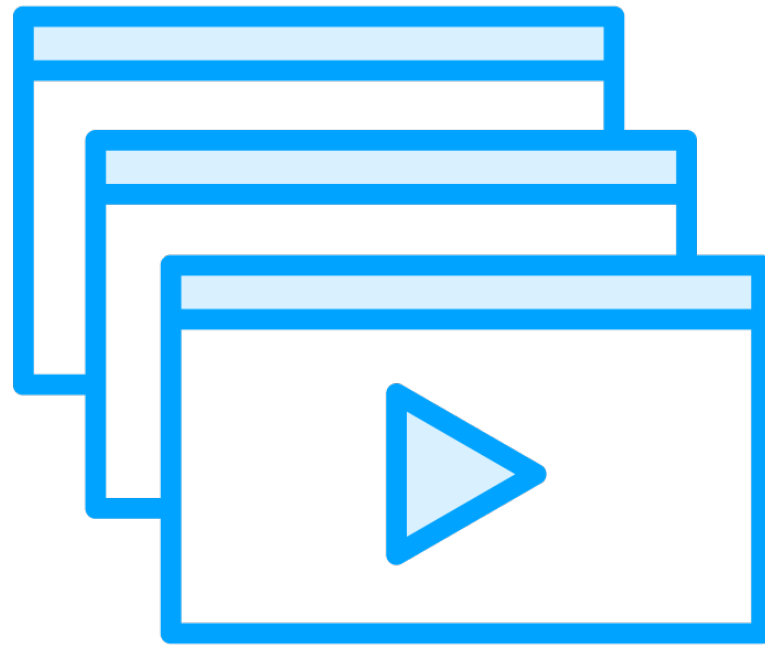
Model training



Sequence generation



Sequence Generation



Model learns to predict the next frame

- In a sequence
- Based on previous frames

Understands both

- The spatial content
- Evolution over time

Essential capability for generating coherent content and realistic video sequences



Types of models used by video-based Generative AI



Recurrent Neural Networks (RNNs)

Type of RNN that can remember
long-term dependencies

Critical for understanding temporal
dynamics in video data

Long Short-Term Memory (LSTM)

Similar to LSTM but with a simple structure

Can model time-series data effectively

Gated Recurrent Units (GRUs)



Generative Adversarial Networks (GANs)

GANs that are adapted for video

Includes mechanisms to capture the temporal progression between frames

Temporal GANs

Divides video into content and motion

Allows for separate generation of static background and dynamic foreground

**MoCoGAN
(Motion and Content
decomposed GAN)**



Transformer

Adaptation of the transformer
architecture but for
video generation

Video transformers

Variant of transformers that deals
with video by incorporating
temporal attention mechanisms

TimeSformer



Other Types of Models

Adapted from image-based VAEs to handle sequences of images

Often used for predicting future frames in videos

Video VAEs

These networks extend the 2D convolutional operations into 3D

Allows the model to process spatial information in frames and temporal information across frames simultaneously

3D Convolutional Neural Networks (3D CNNs)



Other Types of Models

Combine different architectures to leverage their strengths, for example:

- CNNs for spatial processing
- LSTMs for temporal processing

Hybrid Models

Use separate pathways to process spatial and temporal information

Typically, one stream with 2D CNNs for spatial

Another with 3D CNNs or optical flow for temporal aspects

Two-Stream Networks



Other Types of Models

Generate videos by modeling the distribution of video data directly

Often using concepts like optical flow to ensure smooth transitions between frames

Flow-Based

Extended to video, they can generate videos pixel by pixel, frame by frame

They can be computationally intensive for practical use in video generation

**Auto-regressive Models
PixelCNNs and PixelRNNs**



Other Types of Models

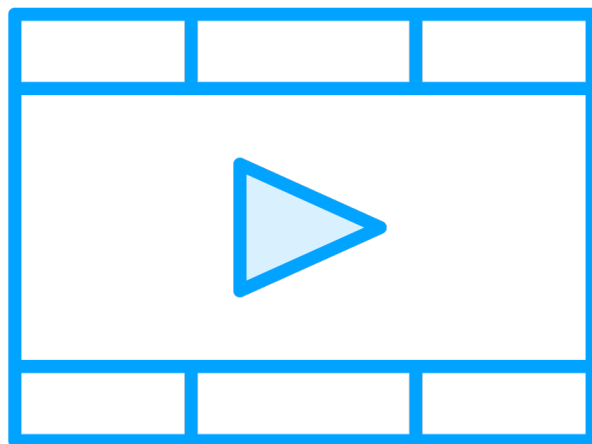
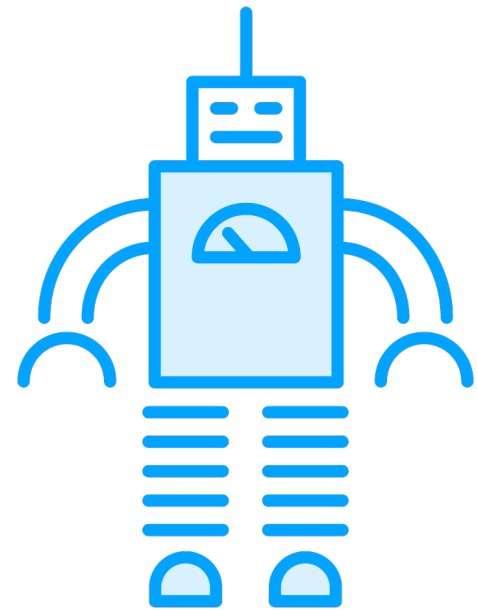
Emerging architecture that tries to encode spatial hierarchies between features

Potentially useful for understanding complex spatial transformations in video

Capsule Networks



Applications of Video-based Gen AI



Each of these models can be applied to different video-based tasks

- Future frame prediction, video super-resolution, video-in-painting, unconditional video generation

Choice of model depends on the task at hand and other factors

Active area of research

Check out OpenAI's Sora





0:19 / 0:59



Prompt: Several giant wooly mammoths approach treading through a snowy meadow, their long wooly fur lightly blows in the wind as they walk, snow covered trees and dramatic sno...
more

0:05 / 0:10



Prompt: A movie trailer featuring th
wool knitted motorcycle helmet, bla
more

A few words of caution on video-based Generative AI





The Future is Here with Generative AI



Thanks for watching!



Xavier Morera

**“What you learn
is yours for life.”**

www.lupo.ai / www.xaviermorera.com

